Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

# Title

Master Thesis

T. Richards

April 7, 2026

Advisors: Prof. Dr. Z. Su, Dr. H. Sun

Department of Computer Science, ETH Zürich

# Acknowledgments

You can use this section to thank whoever you want to in the context of your thesis. This can of course include family and friends, colleagues, your advisor, and in general anyone that provide some form of support. You can also take the opportunity here to give credit to people who may have enlightened you with discussions, and also certainly your co-authors of an academic paper related to the thesis, should there be one.

**Abstract**

We present a randomized algorithm for selecting the median of an unsorted array in $\mathcal{O}(n)$ time with high probability. Unlike classical deterministic methods—notably the median-of-medians algorithm—our approach leverages random sampling to simplify the pivot-selection process and reduce overhead. Specifically, we extract a sublinear sample of the input, sort this sample, and employ its near-median elements as pivots. By bounding $\mathbb{P}[|X - \mathbb{E}[X]| \geq k]$ for appropriate choices of $X$ and $k$, we ensure that the size of the sub-array requiring a full sort remains sufficiently small. A rigorous analysis, based on Chebyshev's inequality, demonstrates that the algorithm's probability of "failure" (i.e., incorrectly bracketed pivots) decays quickly with the input size. In successful executions, the asymptotic cost of sorting the sample and the sub-array is dominated by a single pass through the input, yielding an overall running time of $\mathcal{O}(n)$. While deterministic linear-time solutions to selection exist, our randomized method offers competitive performance in practice, owing to its simplicity and favorable hidden constants. We also discuss how this technique can be combined with deterministic strategies to eliminate worst-case risk.

# Contents

**Contents** iii

**1 Introduction** 1

**2 Background** 2

**3 Related Work** 3

**4 Methodology** 4
- 4.1 Key Challenges . . . 4
- 4.2 Core Ideas . . . 4
  - 4.2.1 Reentrancy Protection . . . 4
  - 4.2.2 Side Effects . . . 5
  - 4.2.3 Side Effects in External Calls . . . 7
  - 4.2.4 Reentrancy as a Side Effect . . . 8
  - 4.2.5 Algebraic Data Types . . . 8
- 4.3 Discussion . . . 9
  - 4.3.1 Retrofitting Solidity . . . 9
  - 4.3.2 Retrofitting Vyper . . . 10
  - 4.3.3 Rust as a Base Language . . . 11

**5 Implementation** 12

**6 Evaluation** 13
- 6.1 Algebraic Data Types . . . 13
  - 6.1.1 Real-world example . . . 14
- 6.2 Reentrancy Protection . . . 16
  - 6.2.1 Real-world examples . . . 16
- 6.3 Side Effects . . . 17
  - 6.3.1 Side Effects in Reentrant Calls . . . 18

**7 Conclusion** 20

**Bibliography** 21

**Dummy Appendix** 22

Chapter 1

# Introduction

This page unintentionally left blank.

Chapter 2

# Background

This page unintentionally left blank.

Chapter 3

# Related Work

This page unintentionally left blank.

Chapter 4

# Methodology

## 4.1 Key Challenges

## 4.2 Core Ideas

### 4.2.1 Reentrancy Protection

As we have seen, reentrancy vulnerabilities are one of the most prevalent, yet easily classifiable kinds of exploits. While the exact attack pattern may vary, it must by definition always involve a nested call into a contract that is already mid-execution. Interestingly, most smart contracts do not even require reentrant calls for their intended purpose. In fact, many contracts that have been exploited via reentrant calls did not require reentrant calls to work at all. In many cases, the issue was simply a lack of adequate protection against reentrant calls.

Given the high financial stakes in this environment, it seems clear that allowing reentrant calls by default is not ideal. Most projects do not use reentrancy at all, and thus developers should not be forced to reason about such cases unless they intend to specifically make use of reentrant calls. This does mean that the language no longer has a one-to-one correspondence with the behaviour of the EVM itself. However, this allows inexperienced developers to simply reason about their own code without needing to worry about the intricacies of the underlying virtual machine.

Existing approaches to reentrancy protection almost always involve adding specific permissions to the entrypoints of the contract. For example, standard practice in Solidity and Vyper is to add a `nonReentrant` modifier to all external functions that do not need to be reentered into. If the default behaviour were simply inverted, we would then expect the entrypoints to be labelled with `reentrant` modifiers where necessary. This is already a vast improvement – any project that does not require reentrancy at all would by definition not have to add any such labels at all, while still being fully protected against reentrancy vulnerabilities. However, there are still some issues. A typical contract will not just make one external call, but make many different calls to various other contracts. It may well be the case that one of these external calls is expected to be reentrant, but that doesn't necessarily mean that it would be safe for *every* external call to then call back into the original contract. Yet with granularity at the level of entrypoints, there is no way to restrict this without the user implementing their own logic on top of the existing protections.

The other natural approach would be to decorate the call site with a `reentrant` label of some kind. Again, this leads to a similar issue - while a reentrant call to one entrypoint

might be safe, any other entrypoint might lead to a bug. Thus, it seems clear that being able to map from a specific external call to a specific entrypoint is necessary.
When considering this from a "safety-by-default" perspective, this also makes sense. The developer should be able to allow exactly the behaviour they need, without allowing any further reentrant calls to be made.

#### Our solution

We propose the following approach: All external calls are not able to reenter the contract by default. The annotation of reentrant behaviour is made at the *external call* site, and must specify the *exact* entrypoint which is intended to be reentered.
For example, let's say we are making an external call to `address` with calldata `data`, and this call should be allowed to reenter the `repay(...)` function. This could be specified as follows:

```
EVM::reentrant_call(address, data, Self::repay)
```

Contrast that to a regular external call, which would be:

```
EVM::call(address, data)
```

The syntax restricts reentrant call path to exactly what the developer would expect during normal execution. It does not impose much overhead – we simply add a reference to the corresponding entrypoint, which the developer surely has in mind anyway. Additionally, the control flow becomes much more obvious: If we consider an auditor reading through the code line-by-line, it is immediately obvious that the above call might lead to a call back to the `Self::repay` function. However, if only the entrypoint itself were annotated, then the transfer of control flow would not be clear.

### 4.2.2 Side Effects

One might say that the entire purpose of smart contracts is to produce side effects on a blockchain. Thus, considering side effects and their consequences is clearly a central part of reasoning about smart contract safety. This is especially relevant for smart contract auditors – given a codebase they have never seen before, they are tasked with ruling out bugs and vulnerabilities within the project, often within only a few weeks. Thus, it is vital for them to be able to reduce the search space as much as possible. We have already shown how this can be done by removing all non-necessary reentrant calls, but the same logic is also valid for side effects.
For illustration, consider something as simple as this:
`f(g(...), h(...))`
What can we say about the side effects produced by this expression? Clearly, `f` must be executed last, since it relies on the results of the other calls. However, the evaluation order of `g` and `h` is ambiguous. Of course, it depends on the language specification – but ideally an audit-oriented language would prevent ambiguous syntax in the first place.
In the specific cases of Solidity and Vyper, the situation is even worse: Not only is the syntax ambiguous, the evaluation order is actually *undefined* in such cases! That is, it is up to the compiler to choose whatever order it deems most convenient. While the evaluation order is almost always left-to-right in practice, there are some exceptions, as demonstrated by the winning submission to the **Underhanded Solidity Contest 2022**. Of course, this specific behaviour is mostly limited to code that is designed to be misleading. However, such intentionally hidden bugs can be just as dangerous as unintended ones.

In this work, we have chosen to focus specifically on *state modifications*. They are likely the most relevant type of side effect for this application, but the principles used can be generalized to many other classes of side effects, such as logging (event emission), state reads, exceptions (reverting of transactions), etc.

### Our solution

Our guiding principles in this case were the following:

- There should be no hidden or implicit side effects.
- The ordering of effects must always be unambiguous.

Let's first look at an illustrating example.

```
contract Counter {
  n: u256
}
impl Counter {
  pub fn foo(self) {
    let val_1 = self.n;

    self.increment();

    let val_2 = self.n;
    assert(val_1 == val_2);
  }
  fn increment(self) {
    self.n += 1;
  }
}
```

Here, we can see that there is a "hidden" side effect when calling `self.increment()`. This may result in the incorrect assumption that `self.n` does not change during this call. While it may be obvious in such a simple example, when considering internal function calls in general, it's not so simple. In a sense, the side effects are "hidden" inside the implementation details of the function being called, so we cannot know the possible side effects without first fully understanding its implementation.
There are two issues at play here – let's first consider the hidden nature of the side effect. If we want to avoid such cases, the developer must somehow specify that `self.n` may change when `self.increment()` is called. There are many potential ways of doing this, but for now let's simply surround the relevant statement with a *permission block*:

```
    modify(self.n) {
      self.increment();
    }
```

Note that we would not need this decoration for *direct* side effects, such as the `self.n += 1;` statement later on. This is because the side effect is visible in the syntax, it is not hidden.
This still leaves us with the second issue – when calling a function, we may not know all potential side effects that might be induced. The caller should not be required to understand the precise implementation details of a function. All relevant properties of a function should be provided by its signature. We can amend this by requiring the `increment` function

to declare that it might modify `self.n`. That way, we encapsulate the information about potential side effects behind the signature of the function.

```
  fn increment(self) modifies(self.n) {
    self.n += 1;
  }
```

In this case, we have used a different keyword for the function-level declarations as opposed to the block-level permissions. This is mostly to make clear that they do not mean the same thing – `modify` is used to annotate *indirect* side effects occurring within the body of a function, while `modifies` is an element of the function signature, declaring the possible side effects that may occur when the function is executed. Note that is likely that this choice of syntax would be confusing to new developers. However, the main purpose here is to demonstrate this as a proof of concept, the specific syntax could always be changed.

### 4.2.3 Side Effects in External Calls

There is one obvious limitation to dealing with side effects in this way – what happens if we don't know the details of the function being called? This is an unavoidable situation when a contract is makes an external call into untrusted code. As soon as the control flow transfers to arbitrary code, the compiler is unable to enforce its own rules regarding side effects. This makes it impossible to enforce properties about *external* state. In fact, it is even difficult to reason about *internal* state at all, due to the possibility of reentrant calls. This is where our first core feature comes to the rescue.

```
contract Counter {
  n: u256
}
impl Counter {
  pub fn foo(self) modifies(self.n) {
    let addr: Address = EVM::caller();
    let data: Bytes = Bytes::empty();
    modify(self.n) {
      EVM::reentrant_call(addr, data, Self::increment);
    }
  }
  pub fn increment(self) modifies(self.n) {
    self.n += 1;
  }
}
```

As it turns out, we have all the information we need – every external call site must specifically declare which functions it may reenter. As we saw before, every function must now declare its possible side effects. These two facts let us guarantee which internal side effects may happen during any external call *at compile time*. This type of guarantee would not otherwise be possible without the use of sophisticated static analysis to rule out reentrancies.
Thus, it becomes even simpler to reason about reentrancy vulnerabilities. Not only do we know exactly what entrypoint is being called, it is also immediately clear at the call site which state variables might be modified in the nested call, despite the external call executing arbitrary code. In fact, if the side effect declarations were extended to also include state reads, we could also much more easily rule out read-only reentrancy bugs as well.

### 4.2.4 Reentrancy as a Side Effect

Ignoring state modifications for a moment, consider the following example:

```
contract Counter {
  n: u256
}
impl Counter {
  pub fn foo(self) {
    let val = self.n;
    self.bar();
    self.n += 1;
  }
  pub fn bar(self) {
    EVM::reentrant_call(addr, data, Self::foo);
  }
}
```

As previously mentioned, we want to avoid hidden side effects in the code. There is another hidden effect at play here – from the perspective of the `foo` function, it is not clear that it may be called in a reentrant context. This is an especially dangerous situation, since by definition there will be a possible overlap between the effects produced by the reentrant call and those in the top-level call. Even if the reentrant call were made to some other entrypoint, it is still vital to carefully consider what exactly may happen during the reentrant call. Therefore, we treat reentrant calls the same way as the state modifications – every function signature must declare its possible reentrancies, and any indirect reentrancies within a statement must be surrounded by a permission block.

```
  pub fn foo(self) reenters(Self::foo) {
    let val = self.n;
    reenter(Self::foo) { self.bar(); }
    self.n += 1;
  }
  pub fn bar(self) reenters(Self::foo) {
    EVM::reentrant_call(addr, data, Self::foo);
  }
```

The overhead induced by this requirement is quite small compared to the state modifications, since reentrant calls are much less common. But we will discuss the overhead of the effect annotations in more detail later on.
There is one edge case we have neglected to address so far. How can we deal with multiply reentrant calls? What about a recursive function reentrantly calling itself? In general, this is an exceedingly rare occurrence. In fact, it seems unlikely for any use case to require nested reentrancies at all. Thus, it would be reasonable to simply disallow this case from occurring at all. Even so, there is no real barrier to reasoning about nested reentrancies with the same logic we have used so far. The caller of such a function must simply declare all the possible further reentrancies, which may include the calling function itself.

### 4.2.5 Algebraic Data Types

TODO

## 4.3 Discussion

To briefly summarize, the three core features we propose are:

- Specifying exact call paths for reentrant calls.
- Explicit side effects (state modifications, reentrant calls).
- Algebraic Data Types

There are several ways these features could be made available to developers as conveniently as possible. The most obvious solution would be to modify an existing widely-used language such as Solidity or Vyper. However, this would present significant challenges, as we will see in a moment. Most other projects targeting EVM are significantly out of date, and often have their own unique feature sets which do not compose well with ours. Additionally, there is little benefit in modifying a language that is not already being used, since the language will have been designed with other goals in mind.
Unfortunately, that seems to indicate the necessity of designing a new language which specifically fits our desired features. In the interest of making it easy to adopt, it should be based closely on an existing well-known general-purpose programming language. As you may have surmised given the code examples we have given so far, the language chosen in this case is Rust.

### 4.3.1 Retrofitting Solidity

Solidity is the oldest, most widely used programming language for EVM. As such, there is also a large incentive not to break backwards compatibility. While there have been breaking changes in the past, they are quite rare. In fact, since the release of Solidity v0.8 in December 2020, no more breaking changes have been made at all.

#### ADTs

Adapting Solidity to allow ADTs would not present too much of a challenge. Assuming they were handled similarly to Rust, one could simply extend the concept of a Solidity `enum` to include fields. Although one might have to reconsider the syntax of a `switch` statement, or alternatively add some new syntax to enable variable bindings when pattern matching. Of course, great care needs to be taken to ensure backwards compatibility not only in the syntax, but also in the ABI encoding of types and function calls, since newly deployed contracts should still be able to interact with all existing contracts.

#### Reentrant Call Paths

As long as we only require reentrant external calls to declare their corresponding entrypoints, this is again just a small syntactic change to the language. However, the `reenters(...)` declaration in function signatures does not play well with Solidity's contract inheritance and virtual functions. This is not necessarily a deal-breaker, but does present some real obstacles. For example, how could one define an overridable function in an abstract contract, such that it is allowed to reenter a function in an implementing contract? There are solutions, but they are all work-arounds of some kind. Additionally, one would have to be able to differentiate between overloaded functions, and between the functions defined on the current contract versus those on its super-types (i.e. `reenters(super.foo)` vs. `foo`).
What's more, Solidity also features function pointers. One would either have to prevent

creating pointers to reentrant functions entirely, or extend the function pointer type to also include potential reentrancies. However, it would not be generalizable – if we wanted to pass `function() reenters(this.foo)` and `function() reenters(this.bar)` as arguments to the same function, we would need to be able to generalize over the function to be re-entered. Solidity has no notion of generics or type variables, so this would not be a natural extension to the language.

#### Effects

Similar to the reentrancy annotations, the main obstacles here are provided by contract inheritance, as well as function pointers. The difficult questions that would need to be answered are:

- How can an abstract contract specify a modification to a state variable of the implementing contract?
- How can we extend the notion of function pointers to naturally include their effects?

Again, we believe these to be prohibitively large obstacles to extending Solidity with our proposed features.

### 4.3.2 Retrofitting Vyper

The design of the Vyper language is much simpler by comparison. It is closely based on Python, but intentionally restricts its feature set so as to remain as simple as reasonably possible. Due to its widespread use, the language is similarly tied up with the responsibility of maintaining backwards compatibility, although less so than Solidity. Thanks to the simplicity of the language, there are fewer obstacles to implementing our proposed features in the language. However, the philosophy behind Vyper (and Python) is more about making the language easy to use, and reducing developer overhead. For that reason, it seems unlikely that such features would ever be adopted by the Vyper compiler team. Even so, we will briefly discuss the viability of each feature in detail.

#### ADTs

There is no real barrier to implementing ADTs and pattern matching in Vyper, since Python already has support for such types. One slight issue is that Vyper does have reference types, but when passing these types to functions they are implicitly copied. This incentivizes less abstraction and shallower call hierarchies. As such, top-level code that involves many ADTs would be littered with pattern matching case distinctions, rather than "hiding" such functionality within methods defined on the type itself. This makes the feature much less attractive to use, especially since many of Vyper's advantages are provided by its simplicity and straight-forward code.

#### Reentrant Call Paths

Although Vyper does not prevent reentrancies by default, it is likely for this to be changed in the near future. However, due to the desire for simplicity, this will be handled via a `@reentrant` modifier on the entrypoints. For this reason, it again seems like the restrictive nature of our feature set is not a good fit for Vyper, even though there are not necessarily significant barriers preventing such an implementation.

#### Effects

Vyper provides modularity through composition. This avoids the problematic function overriding and overloading we encountered with Solidity. The language also does not feature function pointers, which was the other major road-block. Thus, the annotation of side effects would actually fit quite well into the language design, especially since it already has annotations for function declarations. However, *requiring* the side effect annotations to be provided by the developer would go against the language's philosophy, and present a huge breaking change to all existing code. Thus, it would be exceedingly unlikely for this kind of change to be adopted or well-received by existing users of the language.

### 4.3.3 Rust as a Base Language

As demonstrated by Vyper, it can be advantageous to design a niche language closely on an existing general purpose language, so as to ease adoption for new developers. Initially, we had planned to design a functional language, likely syntactically similar to Haskell. However, the decision was made early on to instead make the language imperative, since functional languages tend to be more difficult to learn and are not widely used. Rust was the next natural choice, since it features ADTs and pattern matching natively. In fact, there are not many mainstream imperative languages that do support pattern matching. The somewhat low-level nature of Rust also is a benefit, since we do want to give the developer control over what operations are happening, most notably memory allocations and storage reads / writes. It is of course still possible to build up many layers of abstraction, for example by defining methods on user-defined types or using traits.

Chapter 5

# Implementation

This page unintentionally left blank.

Chapter 6

# Evaluation

In this chapter we will briefly reintroduce the language design concepts we discussed earlier and see how they fit into simple examples of real-world contracts. These example contracts stem from a widely-used smart contract library maintained by OpenZeppelin. They often only implement a very specific functionality and therefore usually remain quite small and easy to understand. We have reimplemented the identical functionality in our language to compare and discuss the differences.
Afterwards, we will look at some vulnerabilities that were exploited in well-known contracts. After seeing the same functionality reimplemented in our language, we will demonstrate the difficulty of writing code that is vulnerable in the same fashion. Of course, we cannot address all types of vulnerabilities, merely the ones we set out to solve initially.

## 6.1 Algebraic Data Types

When modelling a state machine, it is standard practice to make use of regular enumeration types. However, such a value can only encode a single state, but not contain any auxiliary information. Standard workarounds include using "magic values", i.e., encoding state information via setting a field to a specific predetermined value, as well as defining a "kitchen-sink" struct, namely one that contains all fields for all states at the same time.

For example, ERC20 tokens are specified such that the value `u256::MAX` $= 2^{256} - 1$ represents an unlimited allowance, whereas any other value represents an allowance equivalent to its actual quantity. Now, any read or write to such an allowance must be checked against this magic value, leading to a higher risk of developer mistakes. Here, we compare the "standard" implementation of the ERC20 `transferFrom` function, as specified by the ERC20 token standard:

**Listing 6.1:** Solidity

```
contract ERC20 {
  mapping(address => uint256) private _balances;
  mapping(address =>
    mapping(address =>
      uint256)) private _allowances;

  function transferFrom(
    address from,
    address to,
    uint256 value
  ) external returns (bool) {
    require(from != address(0));
    require(to != address(0));

    address spender = msg.sender;
    uint256 amount = _allowances[from][spender];
    if (amount < type(uint256).max) {
      _allowances[from][spender] = amount - value;
    }

    execute_transfer(from, to, value);
    return true;
  }
}
```

**Listing 6.2:** Ours

```
contract ERC20 {
  balances: HashMap<Address, u256>,
  allowances: HashMap<Address,
    HashMap<Address, Allowance>>,
}
impl ERC20 {
  pub fn transfer_from(self,
    from: Address,
    to: Address,
    value: u256
  ) -> bool {
    assert(from != Address::ZERO);
    assert(to != Address::ZERO);

    let spender = EVM::caller();
    self.allowances[from][spender].spend(value);
    self.execute_transfer(from, to, value);
    return true;
  }
}
enum Allowance {
  Amount(u256),
  Unlimited,
}
impl Allowance {
  fn spend(self, value: u256) {
    match self {
      Allowance::Amount(amount) => {
        amount -= value;
      }
      Allowance::Unlimited => (),
    }
  }
}
```

*Note: In both cases, no underflow check is necessary, because both compilers insert overflow checks on arithmetic operations by default. However, it may sometimes be preferable to check manually in order to produce a more specific error message.*

Defining an ADT in such a manner provides two distinct advantages: First, it becomes much simpler to separate the underlying logic (reducing the allowance by the spent amount) from the contract's functionality (transferring tokens). Second, the type system now makes it impossible to misinterpret an unlimited allowance as a finite one, whereas in Solidity the developer needs to actively remember to check for the "magic value" every time they use an allowance.

Of course, in such a simple example the benefits are dampened by the additional overhead induced by the additional code needed to define of the ADT. However, in more complex cases the advantage becomes much clearer. Furthermore, smart contracts often model processes that can be easily represented by a state machine. This is the perfect application for an ADT, as we will see in the next example.

### 6.1.1 Real-world example

The widely-used OpenZeppelin contract library features a `Governor` contract, which models voting processes. Users can submit proposals, each one of which can be in one of eight different states. Depending on the state, a proposal needs to keep track of different values. For instance, while voting is active, it needs to store the deadline for the end of the voting period, as well as the actual vote counts. This data is stored in the `ProposalCore` struct. Precisely which `ProposalState` the proposal is currently in is dictated by the values stored in the fields. The logic of determining this state is implemented by the `state(...)` function.

**Listing 6.3:** Proposal State Data

```
struct ProposalCore {
  uint48 voteStart;
  uint32 voteDuration;
  bool executed;
  bool canceled;
  uint48 etaSeconds;
}
enum ProposalState {
  Pending,
  Active,
  Canceled,
  Defeated,
  Succeeded,
  Queued,
  Expired,
  Executed
}
```

**Listing 6.4:** Proposal State Logic

```
function state(uint256 id) returns (ProposalState) {
  if (proposals[id].executed) {
    return ProposalState.Executed;
  }
  if (proposals[id].canceled) {
    return ProposalState.Canceled;
  }

  uint256 start = proposals[id].voteStart;
  uint256 now = clock();
  if (start >= now) {
    return ProposalState.Pending;
  }

  uint256 deadline = start + proposals[id].voteDuration;
  if (deadline >= now) {
    return ProposalState.Active;
  } else if (!quorumReached(id) || !voteSucceeded(id)) {
    return ProposalState.Defeated;
  } else if (proposals[id].etaSeconds == 0) {
    return ProposalState.Succeeded;
  } else {
    return ProposalState.Queued;
  }
}
```

Recall that this is a widely-used *library contract*, not just some one-off project! As you can see, even just determining what state the proposal is in becomes exceedingly complex. Instead, it would be much simpler to describe a proposal starting from its states, then only include the fields relevant to each state. This is precisely what can be done with an ADT. As an additional benefit, the compiler rejects reading field values that are not relevant to the current state. For some additional clarity, here is what such an ADT might look like, in Rust syntax. It also demonstrates well that most fields are irrelevant for most states.

```
enum ProposalCore {
    Pending {
        voteStart: u64,
    },
    Active {
        voteStart: u64,
        voteDuration: u64,
    },
    Canceled,
    Defeated,
    Succeeded,
    Queued {
        etaSeconds: u64,
    },
    Expired,
    Executed,
```

```
}
```

## 6.2 Reentrancy Protection

Reentrancy vulnerabilities are one of the most widely-known classes of smart contract bugs. Despite the fact that most smart contract developers are aware of their existence, reentrancies are to this day one of the most common vectors for real-world exploits. Thus, it is clear that the tools provided by the current development ecosystem are insufficient to systematically prevent this type of vulnerability.

In fact, Solidity provides no built-in reentrancy protection tools at all, although its *function modifier* feature provides a convenient way for developers to implement their own protection mechanism. Vyper on the other hand does provide opt-in reentrancy protection, although even this protection was exploited due to a compiler bug.
Note that in both cases, the reentrancy checks are specific to their entrypoints. This means that if an entrypoint is not protected, any subsequent external call made by the contract could potentially reenter any of the contract's entrypoints. If an entrypoint *is* protected, any external call made is still able to reenter all unprotected entrypoints of the contract.

It is trivial for the compiler to infer what kind of check needs to be made based on the entrypoint, as well as only allowing specific external calls to reenter only their desired entrypoints.

Only a minority of smart contracts even require reentrant calls for their core functionality, and even fewer of these require anything more than a single reentrant call path. As such, it seems prudent for most cases to disable reentrant calls by default, and allow the developer to individually define each required reentrant call path individually. If the core functionality of a smart contract must include reentrancies, the developer must consider the safety of such calls very carefully. Hence, it is little additional overhead to require them to specify each call path individually. In fact, it requires every necessary call path to be explicitly considered, so it is virtually impossible for an unexpected reentrant call to happen.

### 6.2.1 Real-world examples

There is an abundance of real-world reentrancy vulnerabilities, although many are either outdated or part of large projects and hence difficult to summarize succinctly.

In October 2022, a new class of reentrancy vulnerability was first described – named a read-only reentrancy. It was long the consensus that reentrancy bugs could only occur if the reentrant call itself modified the contract state. However, simply the fact that a read-only function might return an incorrect intermediate value can be enough to trigger a bug. Most cases of such bugs involve price oracles – that is, contracts whose purpose is to estimate the current exchange price between two or more currencies. It is often possible to manipulate these oracles, although it is usually at great expense. In some cases, it is possible to manipulate the oracle price, then for the oracle to call back to the user. If a third-party contract now relies on the value returned by the oracle contract, it will read the manipulated price and be vulnerable to exploitation. Note that this would not be possible if the oracle contract didn't allow control flow to reenter the read-only function.

In fact, the standard approach to reentrancy protection in Solidity does not even allow protecting read-only functions. Instead, developers must implement two separate checks,

one for regular calls and one for read-only calls. Meanwhile, Vyper has extended its opt-in mechanisms to also include read-only functions as well.

todo: reentrancy examples, compound ERC777

## 6.3 Side Effects

Note: "Side effect" is a very broad term that can be applied to many types of behaviour. In this case, we specifically discuss *state modifications*, but the approach can be extended to almost any other kind of side effect, such as state reads, event emission, external calls, and so on.
In a sense, one could say that the purpose of smart contracts is to cause side effects on the blockchain. In any case, it is quite important for developers to be aware of what side effects are happening in what order. In the first case, hidden or implicit side effects in the code can trigger unexpected behaviour, for example if one state change overwrites another, or if outdated values are used for further computations. In the second case, ambiguities in execution order can also lead to unexpected behaviour, although this is more limited to *intentionally misleading* code, such as the winning submission in the **Underhanded Solidity Contest** in 2022.

The evaluation order of expressions is explicitly stated to be undefined in Solidity, which means that a vast amount of deployed smart contracts are actually inducing undefined behaviour during the compilation process. Of course, this is almost always benign, but still represents a real threat. The Vyper documentation also explicitly states that the evaluation order of arguments to built-in functions is undefined. In other cases, the documentation does not specify anything at all.

Keeping track of state modifications throughout the various call paths of a smart contract is also critical in the case of smart contract audits. If it were at a glance clear that an entrypoint cannot trigger certain state modifications, it would be much easier to rule out certain kinds of vulnerabilities, without having to manually go through each line of code. Additionally, if the syntax is explicitly clear about when side effects can happen, it is much less likely for developers to overlook unexpected state modifications. For example, Vyper only allows state variables to be modified via syntax such as `self.xyz = 5`. Contrast this to solidity, where a statement such as `xyz = 5;` does not make immediately clear whether it is modifying a local variable or a state variable of the contract.

However, in both languages, the syntax of a function call does not explicitly distinguish whether or not state modifications can happen. When reading code, it is impossible to know what side effects might be caused by a function call without fully understanding the function's body. In a sense, this is equivalent to the concept of *information hiding*, which means that a caller should not be reliant on the internals of a function, but instead only depend on its signature. Thus, we can require functions to explicitly state any possible modifications in their signatures.
Although this significantly improves the situation, it doesn't entirely solve the problem – if we see a function call like `self.foo()`, it's still necessary to go and look at the signature of the function to determine its declared side effects. Again, this may allow unintended side effects to go unnoticed. If the developer is also required to explicitly allow the side effects at the *call site*, then it becomes immediately clear at a glance what exactly is happening.

```
pub fn foo(amount: u256) -> u256
  modifies(self.last_amount)
```

```
  modifies(self.n)
{
  self.last_amount = amount;

  let count: u256;
  modify(self.n) {
    count = self.increment();
  }

  return count;
}

fn increment() -> u256
  modifies(self.n)
{
  let result = self.n;
  self.n += 1;
  return result;
}
```

As you can see in the above example, only the indirect side effect caused by the call to self .increment() is required to be explicitly declared. In the case of self .last_amoount = amount, it is already clear that the state is being modified without any surrounding declarations.

### 6.3.1 Side Effects in Reentrant Calls

This concept of side effect declaration also pairs very well with the granular reentrancy path declaration detailed earlier. In fact, without such a feature, it would be almost impossible to determine what side effects might occur to the contract state during an external call. However, if the external call must specify exactly which reentrancies are possible, then the compiler can treat the external call as though it were a regular function call, at least concerning side effects. Thus, the call site must be marked with the potential side effects that would be induced if the reentrant call is made.
Note that we also require the possible reentrancy to be declared in the function signature. If the reentrant call itself may trigger further such calls, it should be clear at the call site that this is a possibility. However, it is unlikely for a real-world use case to strictly require nested reentrant calls, so it would also suffice to simply disallow further reentrant calls.

**Listing 6.5:** Ours

```
pub fn foo(
  x: u256
) -> u256
  modifies(self.val)
  reenters(Self::bar)
{
  self.val = x;
  modify(self.val) {
    EVM::reentrant_call(
      Self::bar,
      address,
      data
    );
  }
  return self.val;
}

pub fn bar()
  modifies(self.val)
{
   self.val += 1;
}
```

**Listing 6.6:** Solidity

```
function foo(uint x)
  public
  returns (uint)
{
    value = newValue;
    address.call(data);
    return value;
}

function bar()
  public
{
    value += 1;
}
```

Here, it is again immediately obvious that the new value written to the state variable may be overwritten by the reentrant call. Contrast that to the equivalent Solidity code, where it is not even clear that a reentrant call is possible in the first place, let alone what side effects it may cause. In fact, any of the existing languages for EVM would be equivalent, since their reentrancy protections are declared at the entrypoints, not the call sites.
Of course, this example also makes clear that the strict requirements for side effect declarations result in verbose code. However, we believe that this is a trade-off which is worth the additional overhead, since the stakes are so high.

Chapter 7

# Conclusion

This page unintentionally left blank.

# Bibliography

[1] Manuel Blum, Robert W Floyd, Vaughan R Pratt, Ronald L Rivest, and Robert E Tarjan. Time bounds for selection. *Journal of Computer and System Sciences*, 7(4):448–461, 1973.

[2] Robert Bringhurst. *The Elements of Typographic Style*. Hartley & Marks, 1996.

# Dummy Appendix

You can defer lengthy calculations that would otherwise only interrupt the flow of your thesis to an appendix.

Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

## Declaration of originality

The signed declaration of originality is a component of every written paper or thesis authored during the course of studies. **In consultation with the supervisor**, one of the following two options must be selected:

☐ I hereby declare that I authored the work in question independently, i.e. that no one helped me to author it. Suggestions from the supervisor regarding language and content are excepted. I used no generative artificial intelligence technologies[1].

☐ I hereby declare that I authored the work in question independently. In doing so I only used the authorised aids, which included suggestions from the supervisor regarding language and content and generative artificial intelligence technologies. The use of the latter and the respective source declarations proceeded in consultation with the supervisor.

**Title of paper or thesis**:

**Authored by**:
*If the work was compiled in a group, the names of all authors are required.*

| **Last name(s):** | **First name(s):** |
|---|---|
| | |
| | |
| | |
| | |

With my signature I confirm the following:

- I have adhered to the rules set out in the Citation Guidelines.
- I have documented all methods, data and processes truthfully and fully.
- I have mentioned all persons who were significant facilitators of the work.

I am aware that the work may be screened electronically for originality.

| **Place, date** | **Signature(s)** |
|---|---|
| | |
| | |
| | |
| | |

*If the work was compiled in a group, the names of all authors are required. Through their signatures they vouch jointly for the entire content of the written work.*

[1] For further information please consult the ETH Zurich websites, e.g. https://ethz.ch/en/the-eth-zurich/education/ai-in-education.html and https://library.ethz.ch/en/researching-and-publishing/scientific-writing-at-eth-zurich.html (subject to change).